

How to Control the Result of a Task in C# using TaskCompletionSource

In this article, I am going to discuss **How to Control the Result of a Task in C# using TaskCompletionSource** with Examples. Please read our previous article where we discussed [Only One Pattern in C# Asynchronous Programming with Examples](#).

How to Control the Result of a Task in C#?

So far, we have worked with tasks, and task status depends on an event. For example, if we make an HTTP request or if we make an Async method call, then the status of the task is associated with what happens with the HTTP Request or with the Async Method call, whether it is successful, or there is an exception or the operation is canceled using a cancellation token. With **TaskCompletionSource**, we can create a task for which we are the ones who will control its status, whether it is successful, canceled, or if it's thrown an exception.

Constructors, Methods, and Properties of TaskCompletionSource class in C#:

If you go to the definition of TaskCompletionSource class in C#, you will see the following. You can see it is a generic class.

```
namespace System.Threading.Tasks
{
    ...public class TaskCompletionSource<TResult>
    {
        ...public TaskCompletionSource();
        ...public TaskCompletionSource(TaskCreationOptions creationOptions);
        ...public TaskCompletionSource(object state);
        ...public TaskCompletionSource(object state, TaskCreationOptions creationOptions);

        ...public Task<TResult> Task { get; }

        ...public void SetCanceled();
        ...public void SetException(Exception exception);
        ...public void SetException(IEnumerable<Exception> exceptions);
        ...public void SetResult(TResult result);
        ...public bool TrySetCanceled();
        ...public bool TrySetCanceled(Cancellation_token cancellationToken);
        ...public bool TrySetException(Exception exception);
        ...public bool TrySetException(IEnumerable<Exception> exceptions);
        ...public bool TrySetResult(TResult result);
    }
}
```

Constructors of TaskCompletionSource class:

The TaskCompletionSource class in C# provides the following 4 constructors that we can use to create an instance of the TaskCompletionSource class.

1. **TaskCompletionSource():** It creates a `System.Threading.Tasks.TaskCompletionSource` object.
2. **TaskCompletionSource(TaskCreationOptions creationOptions):** It creates a `TaskCompletionSource` with the specified options. Here, the parameter `creationOptions` specify the options to use when creating the underlying Task.
3. **TaskCompletionSource(object state):** It creates a `TaskCompletionSource` with the specified state. Here, the parameter `state` specifies the state to use as the underlying Task's `AsyncState`.
4. **TaskCompletionSource(object state, TaskCreationOptions creationOptions):** It creates a `TaskCompletionSource` with the specified state and options. Here, the parameter `state` specifies the state to use as the underlying Task's `AsyncState` and the parameter `creationOptions` specify the options to use when creating the underlying Task.

Property of TaskCompletionSource class in C#:

The `TaskCompletionSource` class in C# provides the following property.

1. **Task<TResult> Task { get; }:** It returns the `System.Threading.Tasks.Task` created by this `TaskCompletionSource`.

Methods of TaskCompletionSource class in C#:

The `TaskCompletionSource` class in C# provides the following methods.

1. **SetCanceled():** This method is used to set the underlying Task into the Canceled state.
2. **SetException(Exception exception):** This method is used to set the underlying Task into the Faulted State and binds it to a specified exception. Here, the parameter `exception` specifies the exception to binding to this Task.
3. **SetException(ICollection<Exception> exceptions):** This method is used to set the underlying Task into the Faulted State and binds a collection of exception objects to it. Here, the parameter `exceptions` specifies the collection of exceptions to bind to this Task.
4. **SetResult(TResult result):** This method is used to set the underlying Task into the RanToCompletion State. Here, the parameter `result` specifies the result value to bind to this Task.

Example to Understand How to Control the Result of a Task in C#?

Let us understand this with an example. Let's create a method that will return a task, but it will be a task in which we will control its status. For a better understanding, please have a look at the below image. Here, we have created one method which is returning a Task and taking a string input value. First, we created an instance of the **TaskCompletionSource** class using one of the overloaded versions of the Constructor. Then we are checking the string value using if-else statements. If the input string value is 1 then we are calling the **SetResult** method on the `TaskCompletionSource` instance, this method will set the state of the Task (the task holds by the `TaskCompletionSource` object) to `RanToCompletion`. Next, if the string value is 2, then we are calling the **SetCanceled** method which will set the state of the Task to `Canceled`. If the value is neither 2 nor 3, then we are calling the **SetException** method by passing an exception object which will set the state of the Task to `Faulted`. Finally, we are returning the task by calling the `Task` property of the `TaskCompletionSource` class.

```

public static Task EvaluateValue(string value)
{
    //Creates an object of TaskCompletionSource with the specified options.
    //RunContinuationsAsynchronously option Forces the task to be executed asynchronously.
    var TCS = new TaskCompletionSource<object>(TaskCreationOptions.RunContinuationsAsynchronously);

    if (value == "1")
    {
        //Set the underlying Task into the RanToCompletion state.
        TCS.SetResult(null);
    }
    else if(value == "2")
    {
        //Set the underlying Task into the Canceled state.
        TCS.SetCanceled();
    }
    else
    {
        //Set the underlying Task into the Faulted state and binds it to a specified exception.
        TCS.SetException(new ApplicationException($"Invalid Value : {value}"));
    }

    //Return the task associated with the TaskCompletionSource
    return TCS.Task;
}

```

Next, in order to check whether the task is completed, faulted, or canceled, we are going to use the following three properties of the Task class.

1. **IsCompleted { get; }:** It returns true if the task has been completed; otherwise false.
2. **IsCanceled { get; }:** It returns true if the task has been completed due to being canceled; otherwise false.
3. **IsFaulted { get; }:** It returns true if the task has thrown an unhandled exception; otherwise false.

For this, we are creating the following method. From this method, we are calling the EvaluateValue method. The EvaluateValue method returns one task whose status we managed. Remember, if we are passing 2, then we are throwing an exception. Even if we pass 2, then also it will throw a standard task canceled exception. So, to handle those exceptions we are using the try-catch block and also printing the error message on the console window.

```

public static async void SomeMethod(string value)
{
    var task = EvaluateValue(value);
    Console.WriteLine("EvaluateValue Started");
    try
    {
        Console.WriteLine($"Is Completed: {task.IsCompleted}");
        Console.WriteLine($"Is IsCanceled: {task.IsCanceled}");
        Console.WriteLine($"Is IsFaulted: {task.IsFaulted}");
        await task;
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Exception: {ex.Message}");
    }
    Console.WriteLine("EvaluateValue Completed");
}

```

Following is the Complete Example Code:

```

using System;
using System.Threading;
using System.Threading.Tasks;
using System.Linq;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Enter a number between 1 and 3");
            string value = Console.ReadLine();
            SomeMethod(value);
            Console.ReadKey();
        }
        public static async void SomeMethod(string value)
        {
            var task = EvaluateValue(value);
            Console.WriteLine("EvaluateValue Started");
            try
            {
                Console.WriteLine($"Is Completed: {task.IsCompleted}");
                Console.WriteLine($"Is IsCanceled: {task.IsCanceled}");
                Console.WriteLine($"Is IsFaulted: {task.IsFaulted}");
                await task;
            }
        }
    }
}

```

```

        catch (Exception ex)
        {
            Console.WriteLine(ex.Message);
        }
        Console.WriteLine("EvaluateValue Completed");
    }
    public static Task EvaluateValue(string value)
    {
        //Creates an object of TaskCompletionSource with the specified opt
        //RunContinuationsAsynchronously option Forces the task to be exec
        asynchronously.
        var TCS = new TaskCompletionSource<object>
        (TaskCreationOptions.RunContinuationsAsynchronously);
        if (value == "1")
        {
            //Set the underlying Task into the RanToCompletion state.
            TCS.SetResult(null);
        }
        else if(value == "2")
        {
            //Set the underlying Task into the Canceled state.
            TCS.SetCanceled();
        }
        else
        {
            //Set the underlying Task into the Faulted state and binds it
            specified exception.
            TCS.SetException(new ApplicationException($"Invalid Value : {value}"));
        }
        //Return the task associated with the TaskCompletionSource
        return TCS.Task;
    }
}

```

Now, run the application and enter the value as 1. You will get the following output. Is Completed as true and Is canceled and Is faulted as False.

```

Enter a number between 1 and 3
1
EvaluateValue Started
Is Completed: True
Is IsCanceled: False
Is IsFaulted: False
EvaluateValue Completed

```

Now, again run the application and enter the value as 2. You will get the following output. Is Completed and Is canceled as True and Is faulted as False. As it is canceled so it will throw a task canceled exception that you can see in the Exception message.

```
Enter a number between 1 and 3
2
EvaluateValue Started
Is Completed: True
Is IsCanceled: True
Is IsFaulted: False
Exception: A task was canceled.
EvaluateValue Completed
```

Now, again run the application and enter the value as 3. You will get the following output. Is Completed as True, Is canceled as False and Is Faulted as True. As we throw an exception, so you can that exception message.

```
Enter a number between 1 and 3
3
EvaluateValue Started
Is Completed: True
Is IsCanceled: False
Is IsFaulted: True
Exception: Invalid Value : 3
EvaluateValue Completed
```

So, you can see with TaskCompletionSource, we have complete control over the status of the task.

Example of TaskCompletionSource With Return Value

In the previous example, if you remember we have set null, in the SetResult method. It is also possible to return some value. Let us say we want to return a string value. Then we need to pass the string value to the SetResult method. Apart from this, we need to do two more changes, First, the return type of the method will change from Task to Task<string>, and while creating the instance of the TaskCompletionSource, instead of object we need to pass a string. The following example exactly does the same.

```
using System;
using System.Threading;
using System.Threading.Tasks;
using System.Linq;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Enter a number between 1 and 3");
            string value = Console.ReadLine();
            SomeMethod(value);
            Console.ReadKey();
        }
        public static async void SomeMethod(string value)
        {
```

```

var task = EvaluateValue(value);
Console.WriteLine("EvaluateValue Started");
try
{
    Console.WriteLine($"Is Completed: {task.IsCompleted}");
    Console.WriteLine($"Is IsCanceled: {task.IsCanceled}");
    Console.WriteLine($"Is IsFaulted: {task.IsFaulted}");
    var result = await task;
    Console.WriteLine($"Result: {result}");
}
catch (Exception ex)
{
    Console.WriteLine($"Exception: {ex.Message}");
}
Console.WriteLine("EvaluateValue Completed");
}
public static Task<string> EvaluateValue(string value)
{
    //Creates an object of TaskCompletionSource with the specified opt
    //RunContinuationsAsynchronously option Forces the task to be exec
asynchronously.
    var TCS = new TaskCompletionSource<string>
(TaskCreationOptions.RunContinuationsAsynchronously);
    if (value == "1")
    {
        //Set the underlying Task into the RanToCompletion state.
        TCS.SetResult("Task Completed");
    }
    else if(value == "2")
    {
        //Set the underlying Task into the Canceled state.
        TCS.SetCanceled();
    }
    else
    {
        //Set the underlying Task into the Faulted state and binds it
specified exception.
        TCS.SetException(new ApplicationException($"Invalid Value : {v
    }
    //Return the task associated with the TaskCompletionSource
    return TCS.Task;
}
}
}

```

Run the above code and enter the value as 1 and then you will get the following output. You can observe the Result.

```
Enter a number between 1 and 3
1
EvaluateValue Started
Is Completed: True
Is IsCanceled: False
Is IsFaulted: False
Result: Task Completed
EvaluateValue Completed
```

In the next article, I am going to discuss [Task-based Asynchronous Programming in C#](#) with Examples. Here, in this article, I try to explain **How to Control the Result of a Task in C# using TaskCompletionSource** with Examples. I hope you enjoy this How to Control the Result of a Task in C# with Examples using TaskCompletionSource article.